Institut Gaspard Monge
Licence 3 en informatique
Année universitaire 2025-2026
Université Gustave Eiffel

Rapport de Projet
« Analyseur Syntaxique pour le langage TPC»

```
┌──(braiki-koceila㉿kali)-[~/…/L3/Projets/analyse/Projet_BACKUP]
└─$ ./bin/tpcas -t << 'EOF'
int main(void) {
    int x;
    x = 3 + 4;
    return 0;
}
EOF
Program
├── DeclVars
└── DeclFuncts
    └── DeclFunct: main
        ├── RetType: int
        ├── Params
        └── Body
            ├── DeclVars
            │   └── NodeList
            │       ├── StructType: int
            │       └── NodeList
            │           └── Ident: x
            └── Block
                ├── Assign
                │   ├── Ident: x
                │   └── Add: op
                │       ├── Num: 3
                │       └── Num: 4
                └── Return
                    └── Num: 0
```

Annexe 1 Analyse d'une expression arithmétique

Membres :
BRAIKI Koceila
AMOURA Aylan

1. Introduction

Ce projet consiste à réaliser un compilateur capable de transformer un code source en une structure compréhensible par une machine. L'objectif principal est de développer un outil qui analyse le langage TPC en suivant plusieurs étapes techniques :

1. **Lire un fichier source** écrit en TPC.
2. **Vérifier sa conformité** lexicale et grammaticale.
3. **Construire une représentation intermédiaire** en mémoire sous la forme d'un Arbre Syntaxique Abstrait
4. **Afficher cet arbre** sous forme textuelle ou graphique pour valider la structure du code analysé.

1.1 Outils utilisés

a) **Flex:** Utilisé pour l'analyse lexicale.
b) **Bison :** Utilisé pour l'analyse syntaxique
c) **Langage C :** Utilisé pour définir la structure de l'arbre (struct Node), gérer l'allocation dynamique de mémoire, et faire le lien entre Flex et Bison.

1.2 Architecture générale du programme

Le programme final, nommé tpcas, fonctionne comme ceci :

a) **Entrée :** Le programme lit un fichier .tpc via l'entrée standard.
b) **Analyse Lexicale (tpcas.l) :** Flex identifie les lexèmes et transmet les tokens à Bison. Il gère également le comptage des lignes et des colonnes pour la localisation des erreurs.
c) **Analyse Syntaxique (tpcas.y) :** Bison applique les règles grammaires. À chaque règle reconnue, une action est déclenchée pour créer un nœud d'arbre correspondant (ex: création d'un nœud OP_ADD pour une addition).
d) **Construction de l'arbre (tree.c && tree.h) :** Les nœuds sont assemblés hiérarchiquement (père/fils/frère) pour former l'arbre complet du programme.
e) **Sortie :** Si aucune erreur n'est détectée, l'arbre est affiché (si l'option –tree/-t est activée) puis la mémoire est libérée. En cas d'erreur, un message précis est affiché sur la sortie standard.

1.3 Organisation des fichiers

```
Projet_TPC/
 ├── makefile
 ├── test.sh
 ├── rep/
|        └── RAPPORT.pdf
 ├── src/
|   ├── tpcas.l
|   ├── tpcas.y
|   ├── tree.h
|  └── tree.c
 ├── test/
|        └── tests…
 ├── obj/
└── bin/
```

Figure 2 –

2 Présentation du projet

2.1 Analyse Lexicale (Flex)

Le fichier src/tpcas.l contient l'ensemble des expressions régulières définissant le vocabulaire du langage. Lorsqu'un motif est reconnu, une action en C est exécutée pour :

1. Renvoyer le code du token à l'analyseur syntaxique (ex: return IF;).
2. Stocker la valeur associée si nécessaire .

2.2 Gestion des priorités et stratification

Le fichier src/tpcas.y reçoit les tokens de l'analyseur lexical et vérifier que l'ordre des tokens respecte la grammaire du langage TPC et de déclencher des actions pour construire l'Arbre Syntaxique

2.3 Adaptation et Extension de la structure de l'Arbre (tree.c/h)

a. Enrichir la structure de données Node :

- **Ajout du champ value :** Un pointeur char * a été ajouté pour stocker les noms des identifiants (IDENT), les types (TYPE) ou les valeurs des constantes (NUM, CHARACTER)**.**
- **Ajout du champ lineno :** Pour permettre un diagnostic précis lors des erreurs sémantiques futures, chaque nœud enregistre désormais le numéro de la ligne où il a été créé.

b. Extension de l'énumération des étiquettes (label_t) :

J'ai défini une quarantaine d'étiquettes classées par catégories :

- **Nœuds structurels :** PROG, BODY, DECL_VARS, DECL_FUNCT.
- **Instructions :** INSTR_IF, INSTR_WHILE, INSTR_RETURN, INSTR_ASSIGN.

- **Opérateurs :** OP_ADD, OP_MUL, OP_AND, OP_EQ, etc.

c. Nouvelles fonctions de manipulation :

Pour gérer ces nouveaux champs, j'ai implémenté la fonction **makeNodeVal**. Elle permet d'allouer un nœud tout en lui associant une valeur dupliquée dynamiquement via strdup. La fonction de destruction **deleteTree** a également été mise à jour pour libérer la mémoire allouée par value, garantissant ainsi l'absence de fuites mémoire

d. Amélioration de la visualisation (printTree) :

J'ai conservé la logique d'affichage récursive, mais j'ai amélioré la sortie pour qu'elle affiche les valeurs stockées dans les nœuds.

3. Difficultés rencontrées et Solutions

Le développement de cet analyseur a présenté plusieurs défis, allant de problèmes purement techniques liés au Flex/Bison à des problématiques conceptuelles de modélisation de données.

1. Complexité de la gestion des structures

Le Problème : Initialement, la grammaire ne gérait que des types atomiques (int, char) dont la taille et la nature sont connues d'avance par Bison. Avec les structures, un nouvel identifiant (le nom de la struct) devient lui-même un type. La difficulté était de permettre à Bison de reconnaître struct Point p; sans le confondre avec une déclaration de variable classique ou un appel de fonction.

La Solution :

- J'ai implémenté le nœud STRUCT_DECL pour capturer la déclaration du type (le "moule").
- J'ai utilisé un nœud STRUCT_TYPE pour les déclarations de variables. Ce nœud ne contient pas les champs, mais seulement le nom de la structure de référence.(vérifier si la structure utilisée a bien été déclarée au préalable).

2. Récursivité et accès aux champs imbriqués

Le Problème : La gestion de l'opérateur point (.) pour des accès comme etudiant.date_naissance.annee posait un problème de priorité. l'analyseur peut lire l'expression de droite à gauche, ce qui est syntaxiquement incorrect pour un accès mémoire.

La Solution : J'ai opté pour une récursivité gauche dans la règle **Acces** :

Acces: IDENT | Acces '.' IDENT
Grâce à ce choix, l'analyseur traite d'abord etudiant.date_naissance comme un bloc complet avant de lui appliquer le .annee. Cela crée une structure où le nœud le plus profond est le premier identifiant.

3. Gestion précise de la position (Ligne et Colonne)

Le Problème : Flex fournit nativement yylineno, mais cette variable ne suffit pas pour localiser une erreur au sein d'une même ligne. Pour un utilisateur, savoir qu'une erreur est à la ligne 10 est utile, mais savoir qu'elle se situe précisément à la colonne 24 est indispensable

La Solution :

1. J'ai introduit une variable current_column réinitialisée à chaque saut de ligne (\n).
2. J'ai utilisé la macro YY_USER_ACTION pour calculer automatiquement la position de chaque token :

```
#define YY_USER_ACTION \
  { yylloc.first_line = yylloc.last_line = yylineno; \
    yylloc.first_column = current_column; \
    current_column += yyleng; }
```

4. Extension des fonctionnalités de l'arbre : makeNodeVal

Le Problème : Le module tree initial ne permettait que de créer des nœuds "vides" (sans contenu textuel). Pour un compilateur, un nœud de type IDENT sans le nom de la variable ou un nœud NUM sans sa valeur est inutilisable.

La Solution : J'ai modifié la structure Node pour y intégrer un champ value (char*) et j'ai créé la fonction makeNodeVal.

Difficulté technique : Il a fallu gérer l'allocation dynamique avec strdup pour s'assurer que la valeur persiste même après que Flex ait changé de token. Cela a impliqué une mise à jour de deleteTree pour libérer cette mémoire et éviter les fuites .

5. Pollution par le formatage et les commentaires

Le Problème : Le scanner Flex a rencontré des erreurs "unrecognized rule" . Après des heures de recherche, le problème ne venait pas de la logique mais du formatage du fichier .l.

1. **Tabulations et Espaces :** Des espaces insécables étaient interprétés comme des caractères illégaux par Flex.
2. **Commentaires :** L'utilisation de // dans la section des options de Flex provoquait des erreurs de génération du code C.
3.

La Solution : J'ai dû "nettoyer" le fichier en remplaçant tous les commentaires par le style C classique /* ... */. Cette difficulté m'a appris que les outils de compilation sont extrêmement sensibles à la pureté du fichier source.

6. La Stratification de la Grammaire (Priorités)

Le passage d'une grammaire simple à une grammaire stratifiée est une étape délicate.

- **Le problème :** Sans stratification, une expression comme 3 + 4 * 5 pourrait donner (3+4)*5 = 35.

- **La difficulté :** Il faut créer autant de symboles non-terminaux qu'il y a de niveaux de priorité (Exp, TB, FB, M, E, T, F). Cela rend le fichier tpcas.y beaucoup plus long et complexe à lire, car chaque règle doit renvoyer son résultat au niveau supérieur. C'est un travail de précision où une seule erreur de redirection casse toute la logique arithmétique.

4. Conclusion

Ce projet m’a beaucoup aidé apprendre à créer un compilateur. Au début, c’était assez difficile de comprendre comment Flex et Bison travaillent ensemble, mais j’ai fini par réussir.

La partie la plus dure a été de gérer les structures. J'ai passé beaucoup de temps sur l'opérateur "point" (.) pour que l'on puisse accéder aux variables à l'intérieur des structures. J'ai aussi appris qu'il faut faire très attention aux petits détails, comme les espaces ou les commentaires, parce que Flex est très sensible.

Finalement, mon programme fonctionne bien. Il arrive à lire le code source et à afficher un arbre propre. Ce projet m'a donné plus de confiance pour programmer en C et pour mieux comprendre la logique des langages informatiques.

5. Annexes

```
┌──(braiki-koceila㉿kali)-[~/…/L3/Projets/analyse/Projet_BACKUP]
└─$ ./bin/tpcas -t << 'EOF'
int main(void) {
    int i;
    i = 0;
    while (i < 10) {
        i = i + 1;
    }
    return i;
}
EOF
Program
├── DeclVars
└── DeclFuncts
    └── DeclFunct: main
        ├── RetType: int
        ├── Params
        └── Body
            ├── DeclVars
            │   └── NodeList
            │       ├── StructType: int
            │       └── NodeList
            │           └── Ident: i
            └── Block
                ├── Assign
                │   ├── Ident: i
                │   └── Num: 0
                ├── While
                │   ├── GT: <>
                │   │   ├── Ident: i
                │   │   └── Num: 10
                │   └── Block
                │       └── Assign
                │           ├── Ident: i
                │           └── Add: op
                │               ├── Ident: i
                │               └── Num: 1
                └── Return
                    └── Ident: i
```

Annexe 2 – Structure de contrôle et blocs d'instructions

```
┌──(braiki-koceila㉿kali)-[~/…/L3/Projets/analyse/Projet_BACKUP]
└─$ ./bin/tpcas -t << 'EOF'
struct Point {
    int x;
    int y;
};
int main(void) {
    return 0;
}
EOF
Program
├── DeclVars
│   └── StructDecl: Point
│       └── NodeList
│           ├── FieldAccess
│           │   ├── StructType: int
│           │   └── NodeList
│           │       └── Ident: x
│           └── FieldAccess
│               ├── StructType: int
│               └── NodeList
│                   └── Ident: y
└── DeclFuncts
    └── DeclFunct: main
        ├── RetType: int
        ├── Params
        └── Body
            ├── DeclVars
            └── Block
                └── Return
                    └── Num: 0
```

Annexe 3 – Implémentation de l'extension sur les structures

```
┌──(braiki-koceila㉿kali)-[~/…/L3/Projets/analyse/Projet_BACKUP]
└─$ ./bin/tpcas << 'EOF'
int main(void) {
    int x
    x = 5;
    return 0;
}
EOF
Erreur de syntaxique dans la ligne 3 et la colonne 6 : syntax error
```

Annexe 4 – Localisation précise des erreurs de syntaxe

```
┌──(braiki-koceila㉿kali)-[~/…/L3/Projets/analyse/Projet_BACKUP]
└─$ make clean
make
rm -f obj/* bin/tpcas
bison -d -o obj/tpcas.tab.c src/tpcas.y
src/tpcas.y: avertissement: 1 conflit par décalage/réduction [-Wconflicts-sr]
src/tpcas.y: note: exécutez avec l'option « -Wcounterexamples » pour générer des contre-exemples de conflits
gcc -Wall -g -Iobj -Isrc -c -o obj/tpcas.tab.o obj/tpcas.tab.c
flex -o obj/lex.yy.c src/tpcas.l
gcc -Wall -g -Iobj -Isrc -c -o obj/lex.yy.o obj/lex.yy.c
gcc -Wall -g -Iobj -Isrc -c -o obj/tree.o src/tree.c
gcc -Wall -g -Iobj -Isrc -o bin/tpcas obj/tpcas.tab.o obj/lex.yy.o obj/tree.o
```

Annexe 5 – Compilation du projet et gestion du Makefile

```
┌──(braiki-koceila㉿kali)-[~/…/L3/Projets/analyse/Projet_BACKUP]
└─$ ./test.sh
--- TESTS VALIDES ---
01_calcul.tpc : OK
02_struct.tpc : OK
03_struct_imbriquee.tpc : OK
04_algo.tpc : OK
05_fonctions.tpc : OK

--- TESTS ERREURS ---
err01_lexicale.tpc : OK
err02_syntaxe.tpc : OK
err03_decl_locale.tpc : OK
err04_struct_var.tpc : OK
err05_struct_anonyme.tpc : OK
```

Annexe 6 – Validation par tests automatiques